LAPORAN PRAKTIKUM
SISTEM OPERASI

Modul 05 - Operasi File dan Struktur Direktori



NAMA	: Rosyid Al Ansorri
NIM	: L0125065
KELAS	: A
HARI, TANGGAL	: Kamis, 8 Oktober 2026
WAKTU	: 07:30 – 08:20
DOSEN	: Ery Permana Yudha, S.Kom., M.Kom.
ASISTEN	: Benedhictus Kevin Doni (L0124006)
	Daffa Arrivo Aryanto (L0124093)

PROGRAM STUDI S1 INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI DAN SAINS DATA
UNIVERSITAS SEBELAS MARET

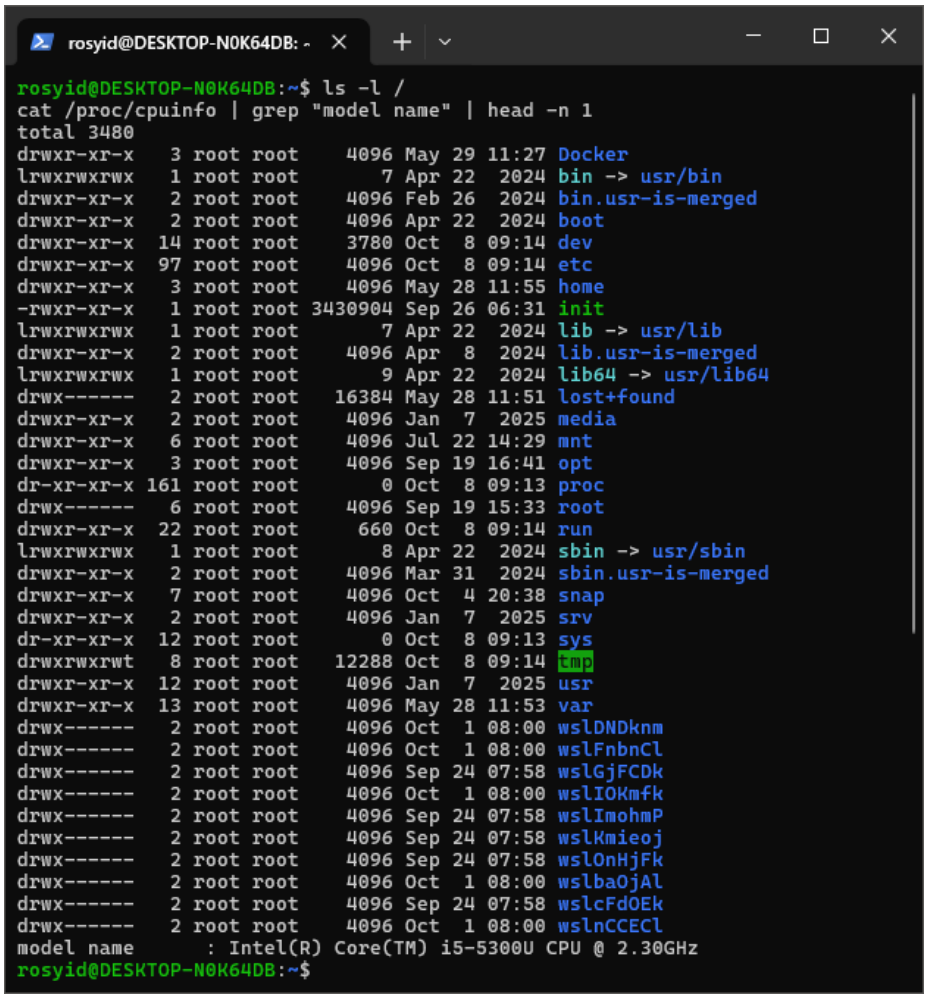
2026

BAB I
LANGKAH DAN PEMBAHASAN

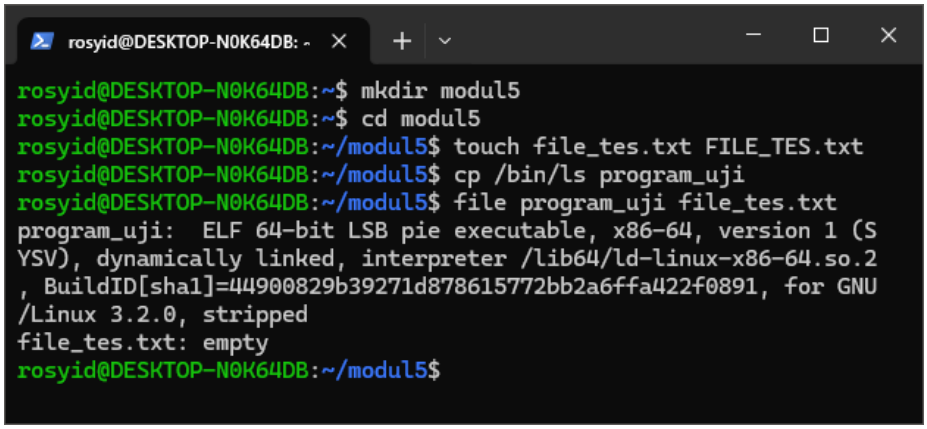
KEGIATAN 1

A. Dokumentasi Kegiatan 1

1. Memeriksa direktori root dan informasi sistem pada `/proc`



2. Menguji aturan penamaan file dan tipe file tanpa ekstensi



3. Catat parameter hasil eksekusi

Aspek Pengamatan	Output Terminal
Output dari `cat /proc/cpuinfo | grep "model name"`	`model name : Intel(R) Core(TM) i5-5300U CPU @ 2.30GHz`
Hasil perintah `file program_uji file_tes.txt`	`program_uji: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=44900829b39271d878615772bb2a6ffa422f0891, for GNU/Linux 3.2.0, stripped` `file_tes.txt: empty`

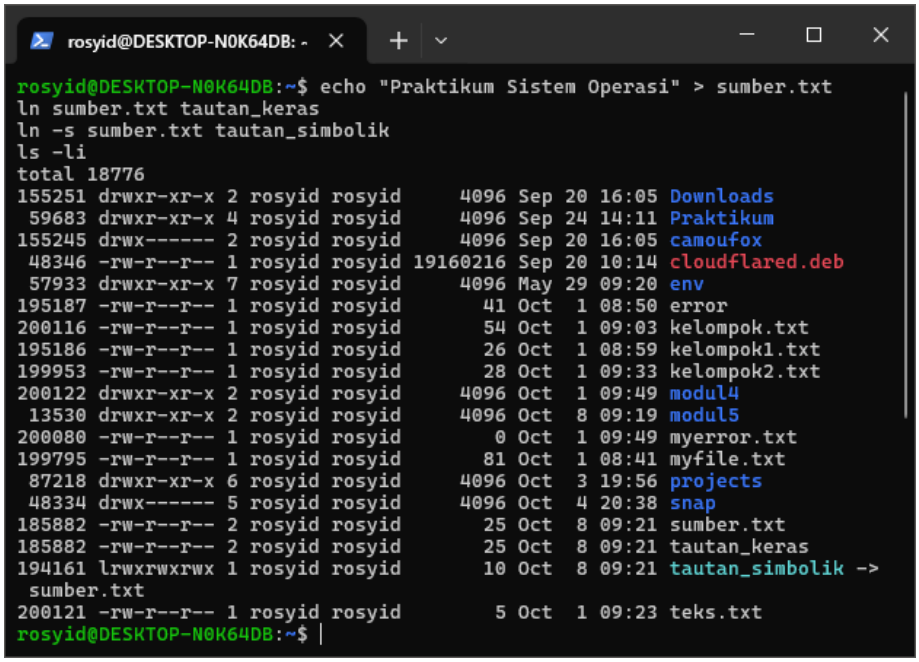
B. Jawaban Analisis Kegiatan 1

1. Linux tidak menentukan tipe file dari ekstensi. Perintah `file` membaca byte-byte awal isi file (magic number), lalu mencocokkannya dengan basis data pola yang dimilikinya. `program_uji` merupakan salinan `/bin/ls` dan dikenali sebagai ELF 64-bit LSB pie executable karena awal isinya memuat header ELF, meskipun namanya tidak berekstensi. Adapun `file_tes.txt` dilaporkan sebagai empty karena ukurannya 0 byte sehingga tidak ada isi yang bisa dicocokkan, sedangkan ekstensi .txt tidak dipakai sebagai petunjuk.
2. Sistem file Linux bersifat case-sensitive, sehingga huruf besar dan huruf kecil dianggap karakter yang berbeda. Nama `file_tes.txt` dan `FILE_TES.txt` pun dibaca sebagai dua nama yang tidak berkaitan. Perintah `touch` kemudian membuat dua file kosong terpisah di folder `modul5`. Pada sistem yang tidak membedakan huruf besar dan kecil, kedua nama itu akan merujuk ke satu file yang sama.

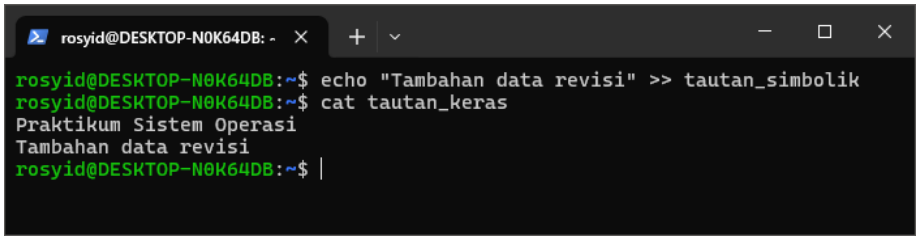
KEGIATAN 2

A. Dokumentasi Kegiatan 2

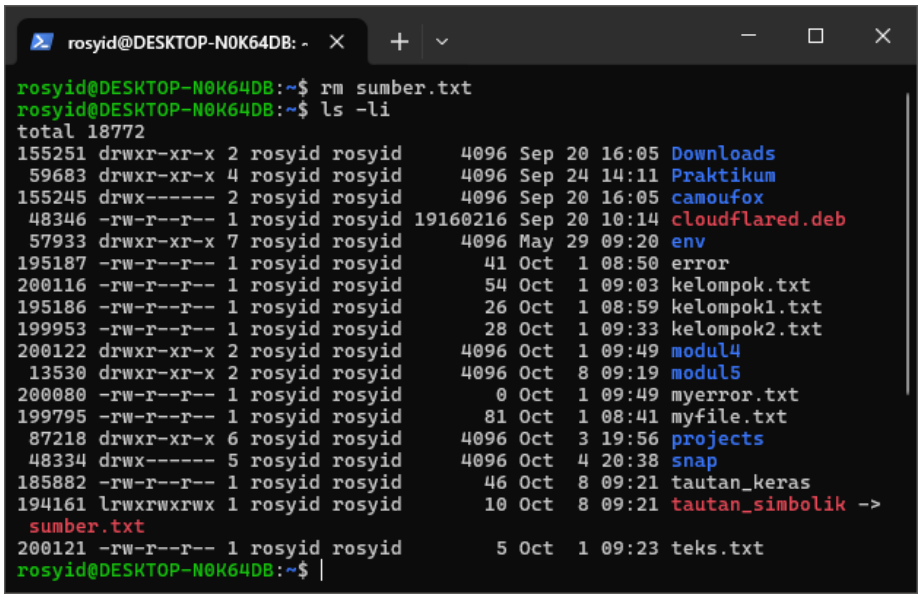
1. Membuat file dan membuat *hard link* serta *symbolic link*

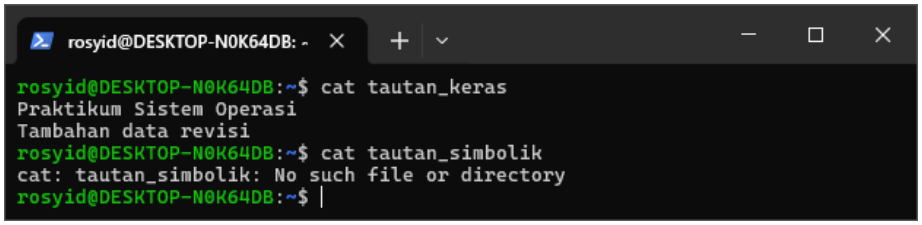


2. Perubahan isi data melalui link



3. Menghapus file sumber





4. Catat hasil pengamatan perubahan file

Aspek Pengamatan	Output Terminal

Output dari perintah `ls -li` pada kegiatan a	`185882 -rw-r--r-- 2 rosyid rosyid 25 Oct 8 09:21 sumber.txt`
	`185882 -rw-r--r-- 2 rosyid rosyid 25 Oct 8 09:21 tautan_keras`
	`194161 lrwxrwxrwx 1 rosyid rosyid 10 Oct 8 09:21 tautan_simbolik -> sumber.txt`

Hasil perintah `cat tautan_keras` pada kegiatan b	`Praktikum Sistem Operasi` `Tambahan data revisi`

Output dari `cat tautan_keras` dan `cat tautan_simbolik` pada kegiatan c	`$ cat tautan_keras`
	`Praktikum Sistem Operasi`
	`Tambahan data revisi`
	`$ cat tautan_simbolik`
	`cat: tautan_simbolik: No such file or directory`

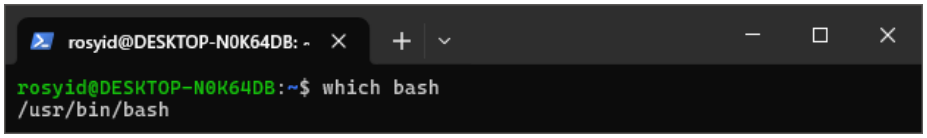
B. Jawaban Analisis Kegiatan 2

1. Pada hard link, `tautan_keras` dan `sumber.txt` memiliki nomor inode yang sama, yaitu 185882, dengan link count 2 pada keduanya. Artinya, keduanya adalah dua nama untuk satu data yang sama di disk. Symbolic link `tautan_simbolik` memiliki inode berbeda, yaitu 194161, bertipe `l` pada kolom izin, dan link count 1. Ukurannya 10 byte, sama dengan panjang teks `sumber.txt`, karena file ini hanya menyimpan jalur menuju file target dan bukan salinan datanya.
2. Perintah `rm sumber.txt` hanya menghapus satu nama dari data tersebut. Link count `tautan_keras` turun dari 2 menjadi 1 dengan inode tetap 185882, sehingga data baru benar-benar dibebaskan ketika link count mencapai 0. Karena itu `cat tautan_keras` masih menampilkan isi file. Sebaliknya, `tautan_simbolik` hanya menyimpan jalur `sumber.txt`. Setelah target dihapus, jalur itu menunjuk ke file yang tidak ada (dangling link), sehingga `cat tautan_simbolik` menghasilkan error No such file or directory.

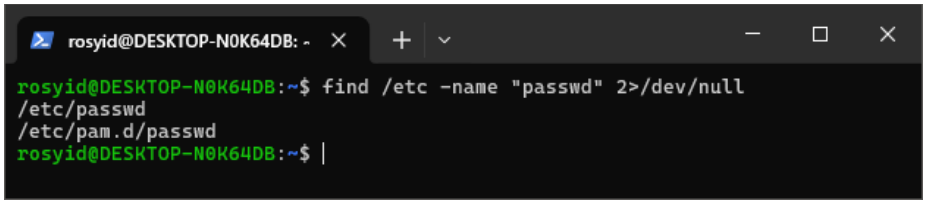
KEGIATAN 3

A. Dokumentasi Instruksi Pengerjaan

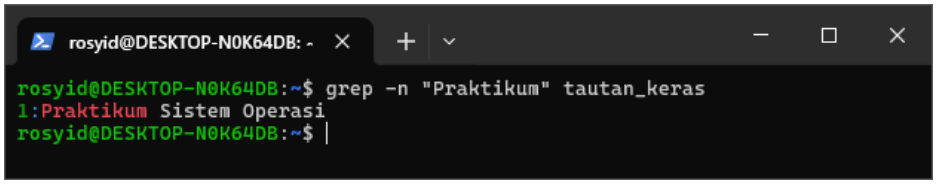
1. Mencari lokasi biner perintah dengan `which`



2. Mencari letak file pada direktori dengan `find`



3. Menyaring isi file dengan `grep`



4. Catat hasil pengamatan perubahan file

Aspek Pengamatan	Output Terminal
Output dari `which bash`	`/usr/bin/bash`
Hasil perintah `find /etc -name "passwd" 2>/dev/null`	`/etc/passwd` `/etc/pam.d/passwd`
Output dari `grep -n "Praktikum" tautan_keras`	`1:Praktikum Sistem Operasi`

B. Jawaban Analisis Studi Kasus

1. Perintah `which` hanya menelusuri direktori yang tercantum pada variabel PATH untuk menemukan lokasi file eksekusi suatu perintah, misalnya `which bash` menghasilkan `/usr/bin/bash`. Perintah `find` menelusuri pohon direktori secara rekursif dari titik awal yang ditentukan dan dapat mencari file apa pun berdasarkan nama, tipe, atau ukuran, tidak terbatas pada perintah yang dapat dieksekusi. Pada `find /etc -name "passwd"`, ditemukan dua file bernama passwd, yaitu `/etc/passwd` dan `/etc/pam.d/passwd`.

2. Opsi `-n` membuat `grep` menampilkan nomor baris tempat pola ditemukan, dengan format nomor baris, titik dua, lalu isi baris. Pada kegiatan c, keluaran `1:Praktikum Sistem Operasi` menunjukkan bahwa kata Praktikum terdapat pada baris pertama file `tautan_keras`.

BAB II
KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Linux mengelola file dan direktori dengan cara yang khas. Tipe file ditentukan dari isinya, bukan dari ekstensi, dan nama file bersifat case-sensitive. Seluruh direktori tersusun dalam satu hierarki yang berpangkal pada root, dan setiap file dikenali melalui nomor inode. Hard link dan symbolic link menunjukkan dua cara menautkan file: hard link berbagi inode dengan file sumber sehingga datanya tetap ada selama masih ada nama yang merujuk, sedangkan symbolic link hanya menyimpan jalur ke target sehingga rusak ketika target dihapus. Perintah which, find, dan grep membantu pengguna menemukan lokasi perintah, file, dan isi file dengan cepat.